

EXION: Exploiting Inter- and Intra-Iteration Output Sparsity for Diffusion Models

Jaehoon Heo, Adiwena Putra, Jieon Yoon, Sungwoong Yune, Hangeol Lee, Ji-Hoon Kim, and Joo-Young Kim

KAIST

Daejeon, Republic of Korea

{kd01050, adiwena.research, j9e8y, imwooong, lhg4294, jihoon0708, jooyoung1203}@kaist.ac.kr

Abstract—Over the past few years, diffusion models have emerged as novel solutions in AI industries, offering the capability to generate diverse, multi-modal outputs such as images, videos, and motions from input prompts (i.e., text). Despite their impressive capabilities, diffusion models face significant challenges in computing, including excessive latency and energy consumption due to the numerous iterations in model architecture. Although prior works specialized in transformer acceleration can be applied to diffusion models, given that transformers are key components, the problem of the iterative nature of diffusion models still needs to be addressed.

In this paper, we present EXION, the first software-hardware co-designed diffusion accelerator that solves the computation challenges of excessive iterations by exploiting the unique inter- and intra-iteration output sparsity in diffusion models. To this end, we propose two software-level optimizations in EXION. First, we introduce the FFN-Reuse algorithm that identifies and skips redundant computations in FFN layers across different iterations (i.e., inter-iteration sparsity). Second, we use a modified eager prediction method that employs two-step leading-one detection to accurately predict the attention score in diffusion models, skipping unnecessary computations within an iteration (i.e., intra-iteration sparsity). We also introduce a novel data compaction mechanism named ConMerge, which can enhance hardware utilization by condensing and merging large and sparse matrices into small and compact forms. Finally, EXION has a dedicated hardware architecture that supports the above sparsity-inducing algorithms, translating high output sparsity into improved energy efficiency and performance. To verify the feasibility of the EXION accelerator, we first demonstrate that it has no impact on accuracy in various types of multi-modal diffusion models, including text-to-motion, -audio, -image, and -video. We then instantiate EXION in both server- and edge-level settings and compare its performance against GPUs with similar specifications. Our evaluation shows that EXION achieves dramatic improvements in performance and energy efficiency by $3.2\text{--}379.3\times$ and $45.1\text{--}3067.6\times$ compared to a server GPU and by $42.6\text{--}1090.9\times$ and $196.9\text{--}4668.2\times$ compared to an edge GPU.

I. INTRODUCTION

Recently, the demand for artificial intelligence (AI) that can imagine and create innovative information like humans has been a driving force in machine learning (ML) research. As a result, diffusion models [12] have emerged as promising solutions in various generative AI industries. For instance, with a simple text prompt as input, Stable Diffusion [35] can generate high-quality images, and Sora [1] can generate vivid and complex videos. Some models can easily generate human motions from text [43] or audio [44] inputs. This versatility highlights their applicability across multiple applications,

making diffusion models a crucial aspect of generative AI technology.

Although diffusion models can create remarkably accurate and detailed outputs, they come with high energy consumption and long latency. This is due to their fundamental characteristic of starting with random noise and progressively removing it to obtain the desired outputs over numerous iterations, which can reach up to 1000 [12]. Furthermore, as diffusion models are primarily based on transformer blocks [45], each iteration involves a significant number of operations. Consequently, their energy consumption and latency are significantly higher than those of other generative models. For example, Stable Diffusion generates images from text prompts with better quality and diversity compared to StyleGAN-XL [37], a popular generative adversarial network. However, our experimental results show that on an identical GPU device, NVIDIA's RTX 6000 Ada [26], the energy consumption and latency of Stable Diffusion are 1546.7J and 11.8s, respectively, which are $23.6\times$ and $9.8\times$ higher than those of StyleGAN-XL.

To solve these problems derived from repeated iterations, a few software (SW)-based approaches [6], [19], [36], [39] that optimize the scheduler of the diffusion model have been proposed to reduce the large number of iterations, i.e., inference steps. However, these fast sampling methods come at the cost of accuracy [53]. Additionally, some of them even require retraining of the pre-trained diffusion model [21].

Another approach to accelerating the diffusion model is using specialized hardware (HW) accelerators to speed up a large number of operations in each iteration without modifying the scheduler aggressively. Existing transformer accelerators can be utilized [9], [20], [32] to speed up computations in each iteration. Most previous transformer accelerators focus on optimizing the query, key, and value (QKV) projection and attention computation using methods such as eager prediction [32] and attention pruning [20]. However, these optimizations do not significantly reduce the energy consumption and latency of the overall diffusion process since they have not exploited the iterative nature of it. To achieve optimal performance, it is necessary to exploit the unique sparsity that exists only in diffusion models.

In this work, we present EXION, the first diffusion accelerator developed to solve the energy consumption and latency problems in diffusion models by exploiting inter- and intra-iteration output sparsity. To this end, we adopt a SW-HW co-

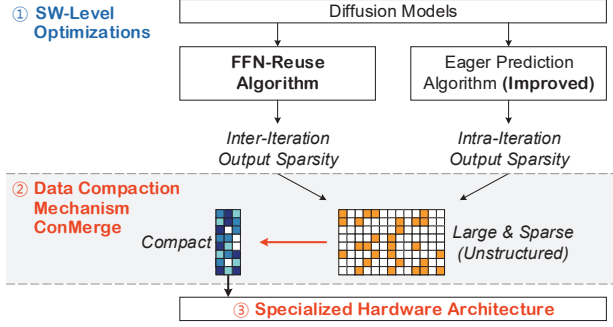


Fig. 1. Overview of EXION Accelerator

design strategy, achieving both output sparsities through SW-level optimizations and fully utilizing them through a data compaction mechanism and dedicated architecture.

Figure 1 shows the overview of EXION. At the SW-level optimization, EXION introduces the FFN-Reuse algorithm, which identifies data redundancy in the matrix multiplication (MMUL) outputs of feedforward neural network (FFN) layers in transformer blocks across different iterations. By utilizing this unique property, the FFN-Reuse algorithm reuses the previous iteration's output data for several subsequent iterations, eliminating the need to compute the inner product for those particular elements and making some output data to be sparse. We refer to this phenomenon as inter-iteration output sparsity. Specifically, it differs from the well-known weight or input sparsities resulting from methods like structured pruning or activation functions, as it arises from data reuse in the FFN's output and can occur independently of them. Moreover, within the execution of a single iteration, EXION has improved the eager prediction algorithm and made it applicable to diffusion models. Similar to inter-iteration output sparsity, the improved method leads to another type of output sparsity during MMUL within the same iteration and the same transformer block, which we term intra-iteration output sparsity.

Although the percentage of resulted output sparsity can dramatically reach up to 97% in our benchmarks, conventional HW, such as GPUs, cannot reduce energy consumption and latency by utilizing it due to the unstructured sparsity in the output matrix. Since GPUs can only manage sparsity in a coarse-grained manner, they are not effective at handling fine-grained sparsity. To address this issue, we propose a data compaction mechanism named ConMerge that enhances HW utilization by condensing and merging large and sparse matrices into small and compact forms. Finally, we developed a tailored architecture for this mechanism, executing operations on the compact matrix forms. Notably, this architecture can handle output sparsity with a flexible yet simple datapath, unlike other accelerators [31], [49], which suffer additional overheads for data feeding to handle sparsity in input or weight data. Additionally, EXION includes an assistance unit to generate control signals for the ConMerge mechanism. Consequently, it successfully translates high output sparsity into improvements in energy efficiency and performance.

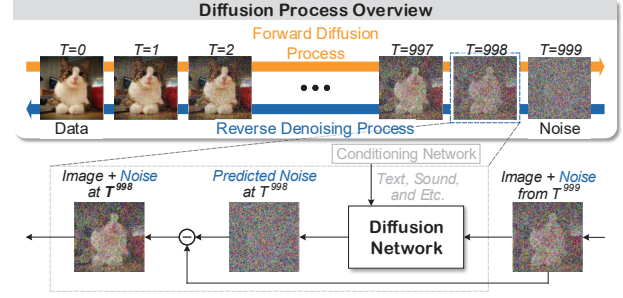


Fig. 2. Overview of Diffusion Models

To verify the efficiency and versatility of EXION, we first demonstrate that its accuracy loss is trivial across various types of diffusion models. Finally, we implemented a cycle-level simulator and compared its performance against the NVIDIA RTX 6000 Ada GPU, verifying that EXION's performance and energy efficiency increased by $3.2\text{-}379.3\times$ and $45.1\text{-}3067.6\times$, respectively. We summarize the contributions of this work as follows:

- We propose EXION, the first software-hardware co-designed diffusion accelerator that solves the computation challenges of excessive iterations by exploiting the unique inter- and intra-iteration output sparsity in diffusion models.
- At the SW-level optimizations, EXION focuses on achieving output sparsity. It introduces the FFN-Reuse algorithm for unique inter-iteration sparsity in FFN layers and proposes an accurate eager prediction method to achieve intra-iteration sparsity in attention computations.
- To efficiently manage sparsity, EXION incorporates the data compaction mechanism ConMerge, which condenses and merges large, sparse matrices into smaller, more compact forms.
- Finally, EXION is equipped with a specialized HW architecture that effectively handles the ConMerge mechanism, resulting in significant improvements in performance and energy efficiency compared to both server- and edge-level GPUs.

II. BACKGROUND AND MOTIVATION

A. Overview of Diffusion Models

Figure 2 shows an overview of the diffusion process, which comprises forward diffusion and reverse denoising processes, using a text-to-image generation example [23]. During the forward diffusion process, the model progressively adds noise to the original input data over numerous iterations, converting the given input to noise. In contrast, the reverse denoising process starts from noise and revives the original input data.

During the training phase, the diffusion network learns to predict noise from input data at the corresponding iteration while repeating the forward diffusion and reverse denoising processes. By subtracting the predicted noise from the input data, the network produces output data with a reduced noise level, which becomes the input for the next iteration. For

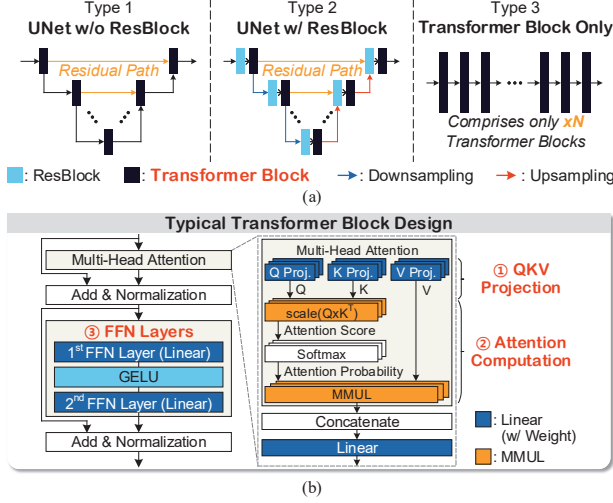


Fig. 3. (a) Diffusion Network and (b) Transformer Block Design

instance, as shown in the figure, at iteration 998 (i.e., when T is 998), the diffusion network predicts the noise at T^{998} using the input received from the previous iteration. Then, it subtracts the predicted noise from that input, resulting in a cleaner image, which will be the input for the next iteration, T^{997} . During the inference phase, only the reverse denoising process is conducted to generate output data from noise. In this paper, we primarily focus on reducing energy consumption and execution time during the inference phase.

When there are conditional inputs, such as text or sound, transformer-based conditioning networks like CLIP [34] and CLAP [7] are executed once to convert these inputs into embeddings. These embeddings are then integrated into the diffusion network during its denoising steps, allowing the generated output to reflect the provided inputs accurately.

As shown in Figure 3 (a), the most important block in the diffusion network is the transformer block, which is included in all types of diffusion models and typically has a large number of operations. Figure 3 (b) illustrates a typical design of the transformer block, where the projection of the query (Q), key (K), and value (V) is first conducted by performing MMUL between the given input and the respective weights. Second, the attention computation is computed by executing MMUL between Q and K^T , applying the Softmax function, and then performing MMUL with V. Finally, after concatenating all the multi-head attention outputs, the FFN layers are executed.

Figure 4 depicts the breakdown of the number of operations in our benchmark models. It demonstrates that the transformer block, including QKV projection, attention computation, and FFN layers, accounts for the highest ratio of operations, ranging from 38% to 100%, leading to the highest energy consumption. Notably, within the transformer block, the FFN layers are generally the most compute-intensive. This is because diffusion models have relatively short token lengths compared to recent large language models, which have significantly longer token lengths and thus require much more

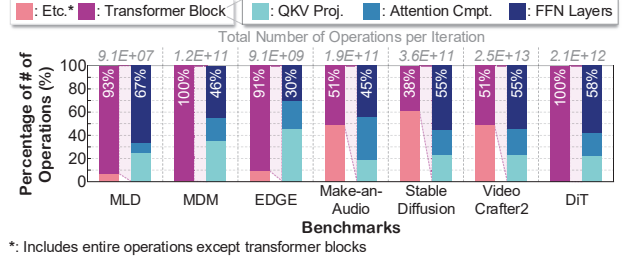


Fig. 4. Number of Operations Breakdown

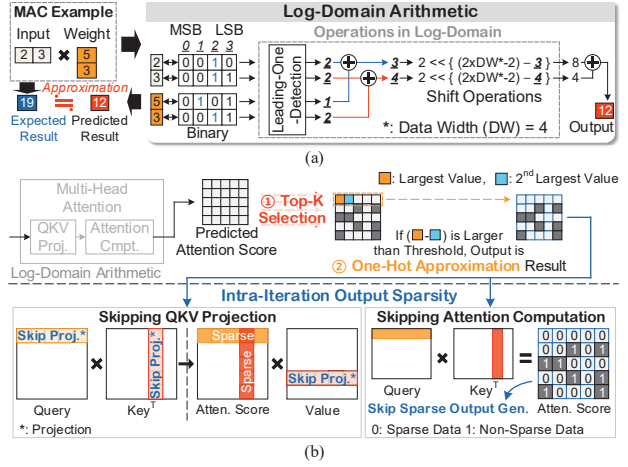


Fig. 5. (a) Log-Domain Arithmetic in EP (b) Intra-Iteration Output Sparsity

computation for QKV projection and attention computation. As a result, as shown in the figure, FFN layers become the main bottleneck with the highest computational overhead in the transformer block of diffusion models, reaching up to 67%. Although prior transformer accelerators have suggested optimization methods, they mostly target the QKV projection and attention computation and are not ideal for the FFN layers in diffusion models. Consequently, we propose a novel method to address the significant overhead of FFN layers in diffusion models, which is described in Section III.

B. Eager Prediction Algorithm

While the number of operations in FFN layers is dominant, this does not mean that QKV projection and attention computation are trivial in diffusion models. To address these efficiently, we adopt an eager prediction (EP) algorithm from the state-of-the-art (SOTA) transformer accelerator [32]. This algorithm first predicts the attention score using simple log-domain arithmetic. Then, using the predicted attention score, it identifies unnecessary output results that can be skipped in more complex real-domain computations.

Figure 5 (a) explains the log-domain arithmetic in detail. It begins with preprocessing by separating the input and weight data of QKV projection into sign bits and absolute values. Then, it identifies the position of the leading-one bit, i.e., the first bit that is set to one from the most significant bit (MSB),

effectively approximating an integer number to a log-domain number. In multiply-and-accumulate (MAC) operations, the multiplication is simplified to an addition followed by a shift operation that converts the log-domain number back to a real-domain number. Finally, it conducts the remaining additions for accumulation, considering the sign bit information.

This straightforward log-domain arithmetic also applies to attention computation to generate the predicted attention score. Since the predicted score serves as the input to the Softmax function, the algorithm leverages the characteristic of the function, where non-important elements are always present in a row of Softmax outputs. As illustrated in Figure 5 (b), we initially perform a top-k selection on each row of the predicted attention score. Values that do not rank within the "top k" are directly assigned to zero, as their influence will be negligible in the subsequent layer after the Softmax function. Moreover, when the difference between the most dominant value and the second most dominant value surpasses a predetermined threshold, the computation for that row can be entirely skipped. This is because the dominant element is already determined, rendering the remaining elements in that row effectively zero. Finally, skipping computation on the negligible elements in the predicted attention score will lead to output sparsity within each iteration during the attention computation, which we term as intra-iteration output sparsity.

As a result, the EP technique achieves intra-iteration output sparsity in attention computation ranging from 20% to 95% in our benchmarks. Additionally, another merit of EP is that when all the elements in a column of the attention score are sparse, we can even skip K and V projection for those columns. Furthermore, when the output is generated directly through one-hot approximation due to one dominant value in a row, the Q projection for that row can be skipped. Finally, in our benchmarks, the EP algorithm skips, on average, 26% of Q projection and 22% of KV projection.

However, the EP method cannot be directly applied to diffusion models, where various transformer blocks exist within a single iteration, and furthermore, those iterations are continuously repeated up to 1000 times. The error due to data approximation accumulates, leading to decreased accuracy. Therefore, we have refined this method by increasing its prediction accuracy to make it applicable to diffusion models. The detailed improvements are delineated in Section IV-D.

III. EXION'S SOFTWARE-HARDWARE CO-DESIGN STRATEGY

To reduce the overhead of numerous operations resulting from excessive iterations, EXION first provides SW-level optimizations consisting of the FFN-Reuse and EP algorithms, which lead to inter- and intra-iteration output sparsity patterns, respectively. Notably, the main novelty of this work and the resulting efficiency gains come from the FFN-Reuse algorithm. This algorithm exploits the iterative nature of the diffusion model, achieving unique inter-iteration output sparsity in FFN layers, which are the main overhead of the transformer block. In this section, we will focus on the major aspects that EXION

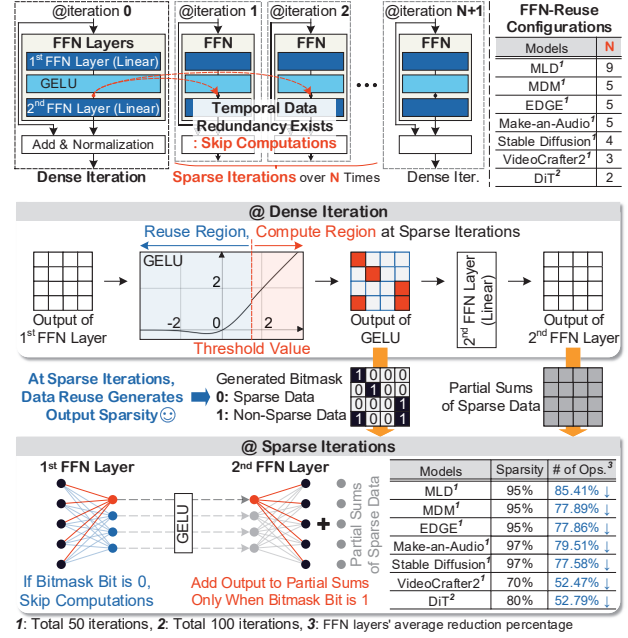


Fig. 6. FFN-Reuse Algorithm for Inter-iteration Output Sparsity

brings, particularly the FFN-Reuse algorithm, followed by the data compaction mechanism, ConMerge.

A. FFN-Reuse Algorithm for Inter-Iteration Output Sparsity

Focusing on the fundamental characteristic of diffusion models that progressively removes noise over numerous iterations, we examined data patterns across these iterations. As a result, we discovered that temporal data redundancy exists across different iterations in FFN layers. Consequently, we propose a FFN-Reuse algorithm that can significantly reduce the number of operations in FFN layers by skipping redundant computations.

Figure 6 describes the algorithm in detail. We first conduct the normal diffusion, i.e., computing all operations required in the FFN layers for a single iteration, which we call a dense iteration. In dense iteration, we also generate the bitmask according to the output of the non-linear layer (e.g., GELU [11] or GEGLU [38]) by comparing the output values with a predetermined threshold. Values larger than this threshold are considered important and need to be recomputed at every iteration, while values smaller than the threshold will be reused for the next N iterations, which we refer to as sparse iterations.

The table in Figure 6 shows that the output sparsity of the 1st FFN layer achieves 70% to 97% on average. The proposed FFN-Reuse leverages this high output sparsity, directly skipping the 1st FFN layer's unnecessary computations. The 2nd FFN layer also skips unnecessary computations by only accumulating the updated values to a partial sum derived from dense iteration. As a result, by continuously executing one dense iteration and N sparse iterations during the entire diffusion process, we can skip 52.47% to 85.41% of operations in FFN layers within our benchmarks.

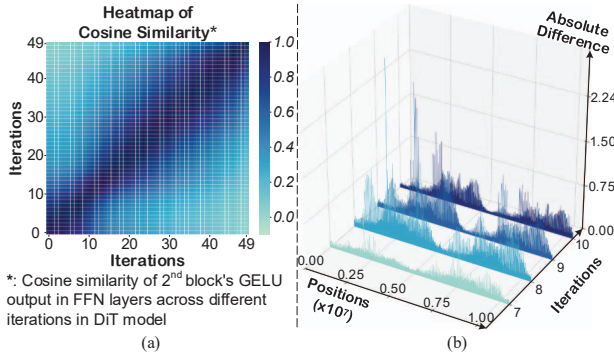


Fig. 7. (a) Heatmap of Cosine Similarity (b) Difference Between Adjacent Iterations

The rationale behind why FFN-Reuse is a feasible solution can be found by analyzing the characteristics of diffusion models. Figure 7 (a) shows the cosine similarity across different iterations in the DiT model [30]. Specifically, it compares the output of the GELU function that exists between two FFN layers. The figure indicates that the cosine similarity between adjacent iterations is high, suggesting the potential for temporal data reuse.

Although most of the outputs can be reused across different iterations, some of the output data cannot be reused. Figure 7 (b) shows the difference in GELU outputs between adjacent iterations. While most of the data shows trivial differences, some points have large differences, indicating computation is required at these points. Notably, the positions where large differences occur are similar across iterations, and we verified that output values at these positions are over the predetermined threshold value we mentioned. As a result, it becomes clear that most of the output data can be reused during sparse iterations while some points need to be kept recomputed. Determining these thresholds, which vary across iterations and transformer blocks, does not require additional training. We can determine these local threshold values through empirical experiments and apply them during runtime.

In summary, in addition to the intra-iteration output sparsity achieved through EP (see Section II-B), the FFN-Reuse algorithm introduces a novel inter-iteration output sparsity in diffusion models, providing more opportunities to reduce the computation in FFN layers dramatically. Consequently, by utilizing both output sparsities with similar patterns, computations in any diffusion model comprising transformer blocks can now be reduced without the need for retraining.

B. ConMerge Mechanism for Data Compaction

While the number of operations in transformer blocks has been significantly reduced, conventional GPUs cannot efficiently leverage the output sparsity. Since GPUs can utilize structured sparsity well, they cannot translate the unstructured inter- and intra-iteration sparsity into high performance and energy efficiency. Additionally, prior works [31], [49] have addressed fine-grained sparsity in either input or weight data by placing additional networks between the memory and

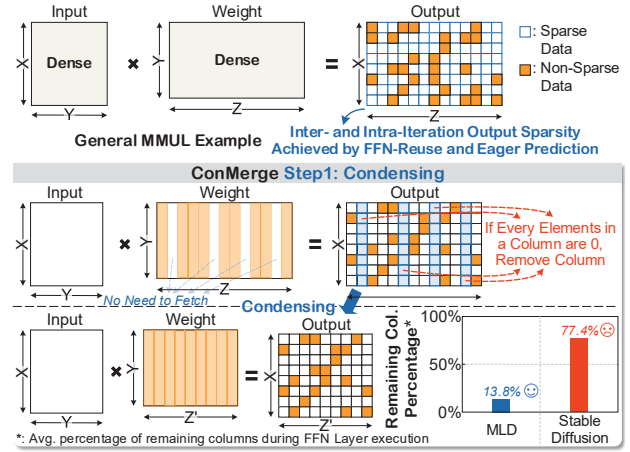


Fig. 8. Process of Condensing

computation engines. Although these methods are efficient, the sparsity in diffusion models is different since it only exists in the output data, while input and weight data remain dense. Therefore, many existing mechanisms and dedicated accelerators [29], [31], [49], [51] designed for input or weight sparsity are not suitable for efficiently handling output sparsity. To solve this problem, we introduce the data compaction mechanism, ConMerge, which consists of condensing and merging.

Condensing. Firstly, we propose a simple but efficient strategy named condensing. As shown in Figure 8, when all elements in a column are sparse, the condensing process removes the corresponding column. This reduces the number of required operations in the MMUL proportionally to the reduction in columns. Moreover, it decreases the required external memory accesses for fetching weight data. The graph in the figure shows the percentage of remaining columns after condensing in MLD [5] and Stable Diffusion models. In the case of MLD, condensing is highly efficient, significantly reducing the number of columns and leaving only 13.8% of the columns.

However, condensing alone can be less effective. For instance, compared to MLD, Stable Diffusion has a relatively large number of rows in the output matrix, making it less likely for all the elements in a column to be zero, even though the overall output sparsity is high. Consequently, the remaining column percentage after condensing is still 77.4%, indicating the need for an additional method.

Merging. To efficiently handle the remaining output sparsity after condensing, we propose a method called merging. Before delving into merging, it is essential to consider the MMUL from an HW perspective. Typically, an HW accelerator running MMUL operations cannot generate all the output data simultaneously and, therefore, employs a tiling strategy. It involves partitioning the output matrix into multiple tiled blocks, which are executed sequentially.

Considering this tiling concept, as illustrated in Figure 9, the first step of the merging is to partition the output matrix

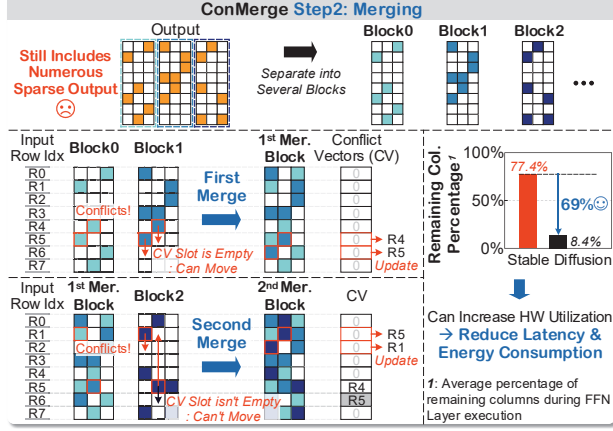


Fig. 9. Illustration of Merging

into multiple blocks tailored to the HW configuration. For example, in the toy HW model shown, the merging separates nine columns into three blocks, with each block having a width of three columns. In the second step, we merge two blocks to create a new merged block, thereby reducing the number of tiled blocks that the HW needs to execute. For example, as shown in the figure, we merge Block0 and Block1 to generate the first merged block. Since both blocks share the same row of the input matrix, merging is straightforward on the row side. However, on the column side, the merged block's columns can originate from either the original weight column of Block0 or Block1. Consequently, every element in the new merged matrix requires tracking its own weight column origin. To manage this efficiently, we incorporate specialized HW components to generate control signals with minimal overhead, with further details provided in Section IV-C.

While merging, conflicts might occur at specific positions when two blocks are not sparse at that position. To solve this problem, we move the elements of the merging block where conflicts occur to other sparse rows within the same column. For instance, if conflicts occur at input rows with indices four (R4) and five (R5), we move the elements of Block1 where conflicts occur to other sparse rows within the same columns, R5 and R6, respectively. Finally, we prepare for the next merging step by updating this information in the conflict vectors (CVs), indicating that in addition to the original input rows R5 and R6, additional input rows R4 and R5 are now required for those rows.

As the last step, we merge the result of the first merged block with the remaining Block2, further reducing the number of tiled blocks. The scenario in the figure's example highlights the necessity of CV. During the second merging phase, conflicts also occur in R1 and R5, but unlike the first merging phase, where all the CV slots are empty, the seventh CV slot is already occupied by R5. Therefore, the mechanism finds another candidate, the R1 row, and finally updates the CV's second slot value as R5.

Although the example illustrates merging with only three

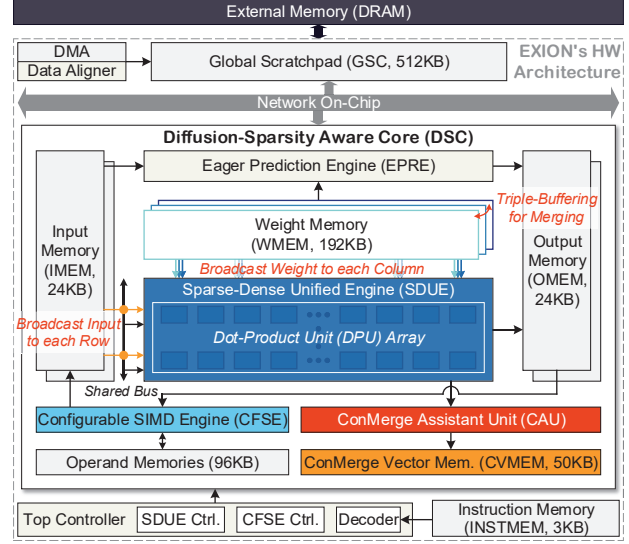


Fig. 10. Hardware Architecture of EXION

blocks, in real scenarios, there are many blocks. If the first merge attempt between Block0 and Block1 fails, the process of merging with Block0 continues with the subsequent blocks, from Block2 to the last block. Once two blocks are successfully merged or all blocks are attempted, the merging moves to the second phase. In the second phase, the merge begins with the first merge result rather than starting from Block0. By generating as many merged blocks as possible, the merging process ultimately succeeds in reducing the remaining columns. For instance, in the Stable Diffusion case, which could not benefit from condensing alone, the percentage of remaining columns dramatically reduced from 77.4% to 8.4%, verifying the efficiency of the merging process.

Consequently, by combining the condensing and merging ideas, the ConMerge mechanism can convert large and sparse output matrices into small and compact matrices, enabling the proposed HW architecture to handle output sparsity with high utilization, thereby reducing latency and total energy consumption. The detailed implementation of the HW architecture and how it addresses the overheads of ConMerge, such as required cycles and HW resource usage, will be described in the following Section.

IV. EXION'S HARDWARE ARCHITECTURE

A. Architecture Overview

Figure 10 illustrates the overall architecture that comprises a top controller with instruction memory (INSTMEM), a diffusion-sparsity aware core (DSC), a global scratchpad (GSC) connected to the DSC via a network-on-chip, and a direct memory access (DMA) that stores/loads data to/from external DRAM.

The heart of the architecture is the DSC, which can run the diffusion model's dense and sparse iterations with an identical core. It is equipped with a sparse-dense unified engine (SDUE) comprising a dot-product unit (DPU) array.

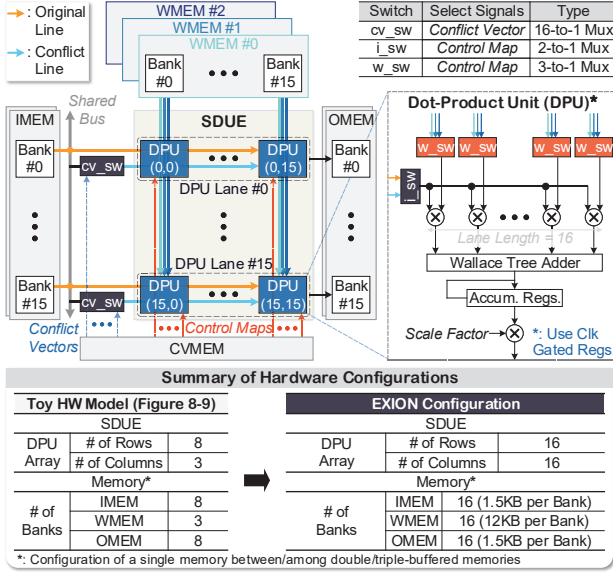


Fig. 11. Design of Sparse-Dense Unified Engine

SDUE can compute the normal dense output matrix and also handle the merged block, which is the result of ConMerge. To this end, input and weight memories (IMEM and WMEM) are double-buffered and triple-buffered, respectively. This buffering scheme is utilized not only to hide the latency of data fetching but also to broadcast the required data to SDUE. Additionally, the DSC includes a ConMerge assistant unit (CAU) with ConMerge vector memory (CVMEM) to generate the control signals. The DSC also includes a configurable SIMD engine (CFSE) with operand memories for accurate computation of special functions such as layer normalization, Softmax, non-linear functions, and residual addition. We design the arithmetic units (ALUs) in CFSE to be configurable, either one-way 32-bit or two-way 16-bit for double throughput. Finally, an eager prediction engine (EPRE) is integrated into DSC to predict the attention score with enhanced accuracy compared to the original EP method.

Operation flow begins by fetching instructions, input, and weight data from the external DRAM to the GSC. Then, the top controller fetches instructions from INSTMEM and, depending on the tiling strategy, unicasts or broadcasts the input and weight to the IMEM and WMEM, respectively. Once the data is ready, the top controller initiates the diffusion process, in which SDUE, EPRE, and CFSE start their computations. During the process, EPRE's latency is mostly hidden by SDUE and CFSE execution due to pipelining schemes. To facilitate this, we adopt an appropriate data mapping and tiling strategy. Moreover, data such as weights and intermediate results are continuously transferred among the DSC, GSC, and external DRAM until the execution is complete.

B. Sparse-Dense Unified Engine

Figure 11 illustrates the design of SDUE, which efficiently computes the most compute-intensive MMUL operations in

the transformer block, regardless of the presence of output sparsity. It consists of an array of DPUs, and each DPU contains integer multipliers, followed by Wallace tree adders [46] and accumulation registers. For normal (dense) MMUL operations without any output sparsity, each DPU receives input and weight data from the corresponding memory banks (e.g., DPU_(0,0) receives input from IMEM bank #0 and WMEM bank #0), executes MAC operations repeatedly and generates the final output of the dot product.

As for the MMUL computation with output sparsity, SDUE maps perfectly to the result of the ConMerge mechanism. In order to achieve this, each row of the DPU array, named DPU lane, is equipped with a conflict vector switch (cv_sw). Consequently, each DPU has two input lines broadcasted from two different IMEM banks. We refer to these lines as the original line and the conflict line, based on the originating IMEM bank being broadcasted. For example, all DPUs within DPU lane #0 directly receive data from IMEM bank #0 via the original line, while the conflict line data can come from any IMEM bank ranging from #0 to #15, as determined by the CV that controls the cv_sw.

Additionally, each DPU is equipped with two switches: a weight switch (w_sw) and an input switch (i_sw). The w_sw allows the DPU to select among three weight lines broadcasted vertically, each corresponding to three WMEMs, WMEM #0 to WMEM #2. Since our ConMerge mechanism allows merging twice, all the WMEMs, which are triple-buffered, are utilized. Weight data in WMEM's bank corresponds to the column of the weight matrix. Therefore, three columns of weight data, each from the bank with the same index in three WMEMs, are broadcasted to each column of the DPU array (e.g., DPU_(0,0) receives weight data from bank #0 of the three WMEMs). On the other hand, the i_sw enables the DPU to choose between the two input lines being broadcasted (original or conflict line). Both switches are controlled by control maps (CMs) generated by the CAU, and with a combination of them, each DPU can flexibly compute any combination of dot product operations with respect to the input and weight being broadcasted.

Consequently, the SDUE enables the realization of the ConMerge mechanism without using large and complex HW resources. This design allows EXION to achieve high utilization and reduce latency. Additionally, clock gating is applied to all the registers in the SDUE's datapath. This strategy addresses any remaining output sparsity after merging, contributing to increased energy efficiency. It is important to note that various accelerators designed to handle fine-grained sparsity in input and weight data often include specialized but complex networks to distribute data to processing engines, which incurs additional latency or area overheads. In contrast, the SDUE broadcasts input and weight data to the DPU array, efficiently supporting the ConMerge mechanism for fine-grained output sparsity handling with relatively small overhead.

C. ConMerge Assistant Unit

In the proposed architecture, the CAU generates the ConMerge vectors that enable the SDUE to address merged blocks

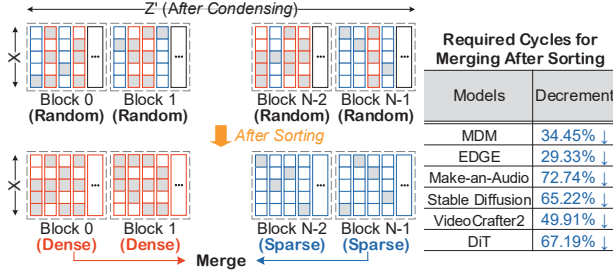


Fig. 12. Sorting Strategy for Faster Merging

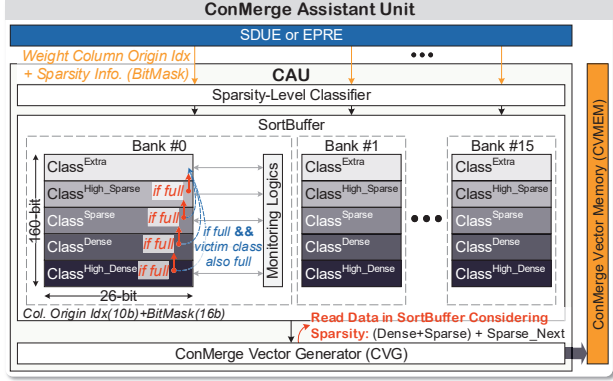


Fig. 13. Efficient Sorting in CAU

by the ConMerge mechanism. To reduce the energy and latency overhead of these additional generation processes, we execute additional sorting before generating the ConMerge vectors. As shown in Figure 12, merging blocks after sorting the columns by the level of sparsity can reduce the total cycles required for the merging process. Compared to merging two blocks with random sparsity, merging two blocks with consideration of their sparsity levels, such as one dense and one sparse, reduces the chances of failure and the need to try merging with other blocks. The figure indicates that this strategy can successfully reduce cycle counts for ConMerge vector generation, ranging from 29.3% to 72.7%. This reduction contributes to lower additional overheads of the ConMerge mechanism. However, implementing naive sorting can incur additional overheads, such as high latency and area. Therefore, we optimize the buffer in the CAU for efficient sorting.

Figure 13 illustrates the efficient sorting method in the CAU. While SDUE computes the FFN layer of dense iterations, the CAU receives the output matrix's original column index and sparsity information represented as a bitmask from each DPU lane. Then, a sparsity-level classifier first counts the number of non-zero bits in the bitmask and decides the sparsity level of each input data, from high_dense to high_sparse. Next, the SortBuffer selects a class and stores the data in the corresponding class based on its sparsity level. During this process, if a class is full, it sends the input bitmask with the column index to the next sparse class, and if that is also full, it sends the bitmask to the extra class. As a result, the bitmask

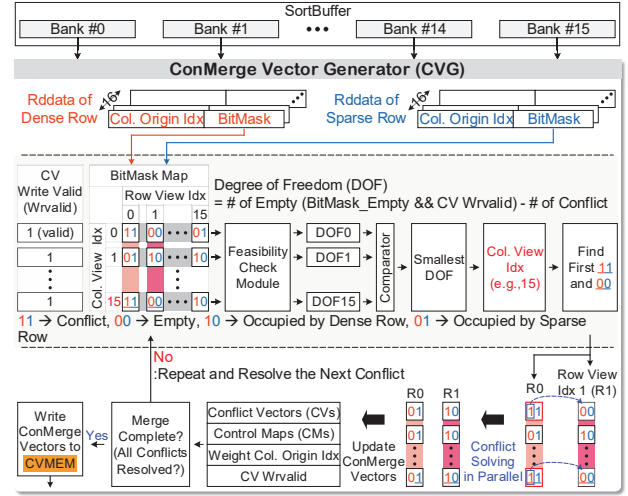


Fig. 14. Detailed Merging Process in ConMerge Vector Generator

is sorted depending on the sparsity level, not completely but in a coarse manner, which is sufficient to increase the success ratio of merging. Additionally, when data in bitmasks are all zero, those inputs are not stored in the SortBuffer, constituting the condensing in the ConMerge mechanism.

Figure 14 shows how EXION generates control signals to support the merging process within the ConMerge vector generator (CVG). Once sorting is complete, the CVG reads the information in the SortBuffer, selecting one row from the most dense and the other from the most sparse. It then integrates the bitmasks from both rows to create a bitmask map. Using this bitmask map, it identifies problematic columns by calculating the degree of freedom (DOF) for each column view index and selecting the one with the smallest DOF. Within that column, the CVG selects two row view indices by finding the first conflict slot and the first empty slot. Then, it resolves conflicts by moving conflicting slots to empty ones in parallel and subsequently updating control signals such as CVs, CMs, and the weight column origin index. These steps are repeated until all conflicts are resolved. After successfully merging two rows, the CVG continues by merging an additional sparse row from the SortBuffer, writing the final output to CVMEM, and proceeding to the next merging operation. Consequently, the efficient sorting method and subsequent control signal generation in the CAU enable EXION to effectively manage the ConMerge mechanism overheads while keeping the CAU minimal in size, accounting for only 0.94% of the DSC area.

D. Eager Prediction Engine

As aforementioned in Section II, the SOTA transformer accelerator has proposed the EP method that predicts the attention score with a simple log-domain algorithm. However, directly applying this scheme to diffusion models incurs critical accuracy loss. Figure 15 illustrates image generation with the DiT, showing that EP's leading-one detection (LOD) produces a poor-quality image and incurs a PSNR drop compared to applying the FFN-Reuse only.

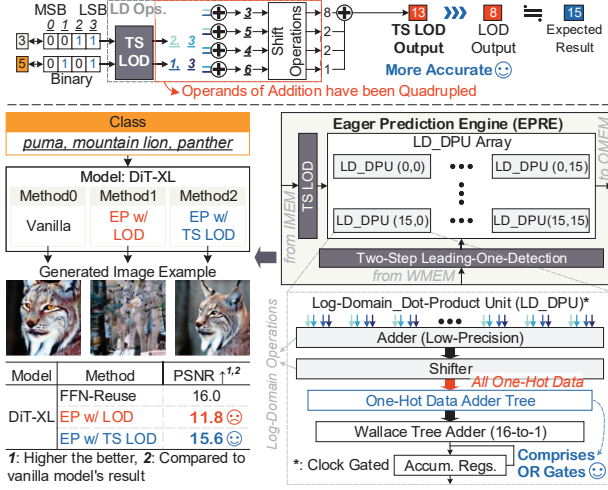


Fig. 15. EPRE with Two-Step Leading-One Detection

To overcome this problem, we execute two-step leading-one detection (TS LOD) in EPRE before sending data to the log-domain DPU (LD_DPU) array. This process first conducts LOD and then detects an additional bit after converting the leading-one bit to zero, finding the positions of two bits in total. We have verified that the PSNR increases from 11.8 to 15.6 with this simple improvement. However, since the operands of addition are quadrupled, the overhead of addition might become a problem with an unoptimized implementation. We exploit the data characteristic in the log domain, where the output of the shift operations is always one-hot data. Consequently, we develop a one-hot data adder tree comprising only simple OR gates in each LD_DPU, successfully reducing the overhead of the TS LOD method.

V. EVALUATION

A. Accuracy Evaluation

Workloads. To demonstrate the flexibility and accuracy of EXION, we selected seven different diffusion models as our workloads, as they encompass all three types of diffusion models (see Figure 3). These models include various applications such as text-to-motion (MLD [5] and MDM [43]), music-to-motion (EDGE [44]), text-to-image (Stable Diffusion [35]), class-to-image (DiT [30]), text-to-audio (Make-an-Audio [13]), and text-to-video (VideoCrafter2 [2]) generations.

Accuracy. Since the rationale for achieving output sparsity in diffusion models is fundamentally based on an approximation strategy, we focused on verifying that the accuracy loss incurred by the proposed EXION is minimal. To this end, we tested the model accuracy in various diffusion models with the given dataset. Additionally, to demonstrate that the proposed solution in EXION can be generally adapted to any diffusion model during inference, we did not conduct any retraining. We also determined values such as the threshold in FFN-Reuse and the top-k selection ratio through empirical experiments.

Table I shows the results of the accuracy evaluation experiments. They indicate that the proposed FFN-Reuse algorithm,

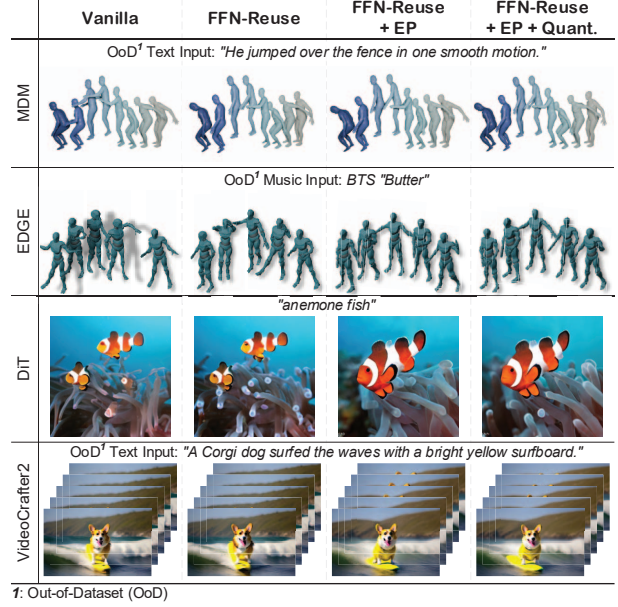


Fig. 16. Visualization of Generated Outputs

which achieves inter-iteration sparsity of 70-97%, has no impact on accuracy. Across all the benchmarks, the evaluation metrics in each model show a minimal difference compared to the results of the vanilla model. This ensures that the FFN-Reuse is a feasible solution and can be applied to other diffusion models beyond our benchmarks.

In addition to FFN-Reuse, adopting the EP method achieves additional intra-iteration sparsity of 20-95%. This approach also shows trivial differences compared to the results of the vanilla model. However, there are exceptions for both MDM and EDGE models. For these two models, one of the two different metrics shows a trivial difference while the other does not. Despite this, Figure 16, which visualizes the generated outputs, demonstrates that the quality of the results remains uncompromised even when one metric indicates poorer accuracy. Moreover, these experiments were conducted using out-of-dataset inputs, except for the DiT model, indicating that EXION's optimizations are applicable to production environments. Additionally, we verified that accuracy is maintained after applying post-training quantization, reducing MMUL operations to 12-bit INT and other operations to either 16-bit or 32-bit INT, aligning with our HW architecture.

B. Experimental Setup

Performance. To evaluate the performance of the EXION accelerator, we developed a custom cycle-level simulator and integrated it with Ramulator [15], a widely used DRAM simulator, to model DRAM latency. For more comprehensive results, we evaluated the performance of EXION against two different settings: the edge GPU (NVIDIA Jetson Orin Nano [24]) and the server GPU (NVIDIA RTX 6000 Ada [26]). To ensure a fair comparison, EXION was

TABLE I
MODEL ACCURACY EVALUATION

Models	MLD	MDM	EDGE	Make-an-Audio	Stable Diffusion	DiT	VideoCrafter2
Task	Text-to-Motion	Text-to-Motion	Music-to-Motion	Text-to-Audio	Text-to-Image	Image Generation	Text-to-Video
Dataset	HumanML3D	HumanML3D	AIST++	AudioCaps	COCO 2014	ImageNet 2012	ECTV
Total Iterations	50	50	50	50	50	100	50

Output Sparsity Achieved by EXION's Software-Level Optimizations

FFN-Reuse	Inter-Iter. Sparsity N [†]	95%	95%	95%	97%	97%	80%	70%
		9	5	5	5	4	2	3
EP ²	Intra-Iter. Sparsity (q_th ³ , k ⁴)	30% (q_th=0.3, k=0.7)	95% (q_th=0.3, k=0.05)	50% (q_th=0.9, k=0.5)	80% (q_th=0.7, k=0.2)	20% (q_th=0.8, k=0.8)	95% (q_th=0.15, k=0.05)	50% (q_th=2, k=0.5)

Accuracy Evaluation Metric

Applied Methods	FID (↓) w/ GT ⁵	R-Precision (↑)	FID (↓) w/ GT ⁵	PSNR w/ Vanil. ⁶ (↑)	PFC (↓)	Beat Align Score (↑)	FAD (↓) w/ GT ⁵	PSNR w/ Vanil. ⁶ (↑)	FID (↓) w/ GT ⁵	IS (↑)	PSNR w/ Vanil. ⁶ (↑)	FID (↓) w/ GT ⁵	IS (↑)	PSNR w/ Vanil. ⁶ (↑)	VQA A(↑)	IS (↑)	PSNR w/ Vanil. ⁶ (↑)
Vanilla Model	0.393	0.754	0.406	-	1.352	0.218	4.618	-	26.63	33.11	-	10.63	265.73	-	59.68	17.06	-
FFN-Reuse	0.401	0.752	0.467	19.22	1.389	0.214	4.932	27.14	26.05	33.04	18.20	14.42	265.73	15.99	58.92	16.83	28.25
FFN-Reuse+EP ²	0.410	0.745	0.968	17.84	2.241	0.195	4.975	26.09	-	-	14.06	-	-	14.60	-	-	27.86
FFN-Reuse+EP ² +Quant. ⁷	0.410	0.744	1.080	17.67	2.411	0.193	5.131	25.72	-	-	13.94	-	-	14.57	-	-	27.56

[†]: # of sparse iterations between two dense iterations, ²: EP w/ TS LOD, ³: Threshold for the difference between the largest value and the 2nd largest value, ⁴: Top-k selection ratio (i.e., k=0.5 selects 50% of the data), ⁵: Ground truth, ⁶: Compute PSNR compared to the vanilla model, ⁷: After post-training quantization (INT mixed precision, 12b for SDUE/EPRE and 16/32b for CFSE)

TABLE II
HARDWARE SPECIFICATIONS OF GPUS AND EXION

Hardware Specifications of GPUs		
	Edge GPU	Server GPU
	NVIDIA Jetson Orin Nano	NVIDIA RTX 6000 Ada
Throughput	40.0 TOPS	91.1 TFLOPS ¹
Memory Bandwidth	68 GB/s	960 GB/s
Power Consumption	-15W	-300W

Comparable EXION Configurations

	EXION ⁴	EXION ²⁴
Number of DSCs ²	4	24
Throughput	39.2 TOPS	235.2 TOPS (=88.2 TFLOPS ³)
Memory Bandwidth	51 GB/s	819 GB/s
Power Consumption	-3.18W	-20.40W

¹: FP32 w/o sparsity, ²: Throughput of a single DSC is 9.8 TOPS, ³: Normalized throughput for equivalent comparison between INT12 and FP32 data formats

instantiated in two distinct configurations, summarized in Table II. To match the peak performance and DRAM bandwidth of the edge GPU, which are 40TOPS and 68GB/s using LPDDR5, we instantiated EXION⁴ with 4DSCs and 51GB/s DRAM bandwidth using LPDDR5. For the server GPU, which has 91TFLOPS and 960GB/s using GDDR6, we instantiated EXION²⁴ with 24DSCs and 819GB/s DRAM bandwidth.

Also, for the ablation study, we evaluated EXION by applying the proposed methods in this paper incrementally. While the $\times$ indicates the number of DSCs, we tested in four different configurations: EXION^{*}_Base without any optimizations, EXION^{*}_EP with EP, EXION^{*}_FFNR with FFN-Reuse, and EXION^{*}_All with both EP and FFN-Reuse.

Power/Area. To estimate the power consumption and area of EXION, we first implemented its design with a single DSC at the RTL level using System Verilog and then synthesized it using Synopsys's Design Compiler [41] with 14nm process technology. We verified that it operates at 0.8V and an 800MHz clock frequency without any timing issues. Values from the synthesized results are used as the power consumption and area usage of EXION. For power modeling of LPDDR5 DRAM, we utilized the energy information provided by the DRAM vendor [14], [17]. Finally, we integrated these results into our custom simulator, accurately measuring the power consumption and area of EXION. Moreover, to measure the power consumption of GPUs, we utilized the NVIDIA-

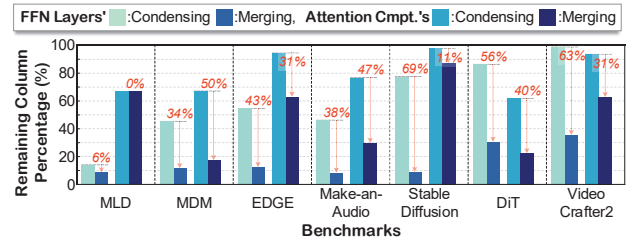


Fig. 17. ConMerge Efficiency

SMI [27] and tegrastats [28] tools.

C. Performance Evaluation

ConMerge Efficiency. Figure 17 shows the efficiency of the proposed ConMerge mechanism, comprising condensing and merging. To verify their efficiency in each step, the figure displays the remaining column percentage in the output matrix of the first FFN layer and the attention score after applying condensing and then merging. For the condensing case, the average percentage of remaining columns in the first FFN layer and attention score is 60.3% and 80.0%, respectively. Although small models such as MLD and MDM can significantly reduce columns with condensing alone, the average value appears high. This is because, in models where the row size of the output matrix is large, there is less chance that all elements in a column are sparse. For instance, although the output sparsity is high at 97% and 70% in Stable Diffusion and Videocrafter2, respectively, 77.4% and 98.6% of output columns in the FFN layer still remain.

Merging solves this issue by significantly reducing these remaining columns. The average percentage of remaining columns in the first FFN layer and attention score achieve 16.2% and 50.0%, respectively, after merging. Notably, for the problematic models, only 8.4% and 35.2% of columns in the FFN layer are left for Stable Diffusion and VideoCrafter2, respectively. These results verify that the proposed ConMerge successfully enables the EXION to handle output matrices with high sparsity.

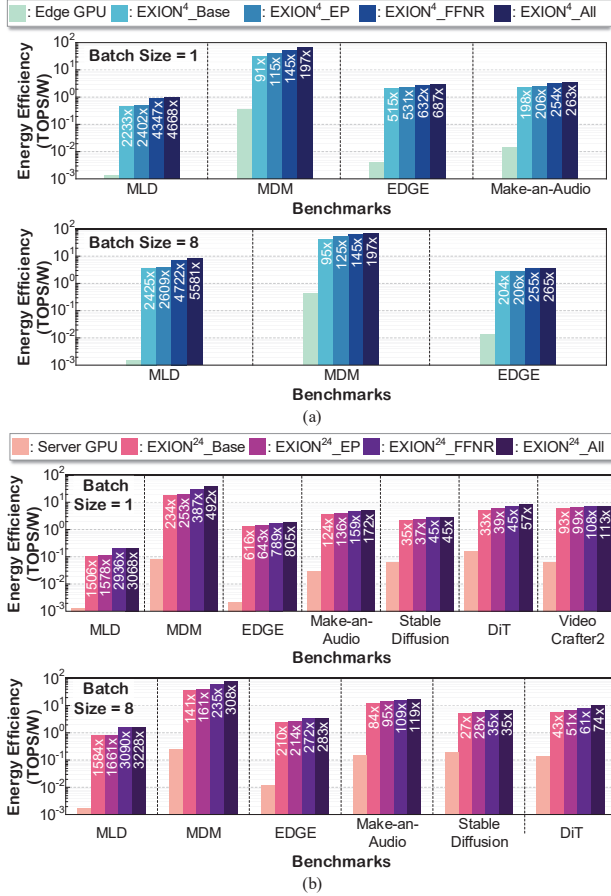


Fig. 18. Energy Efficiency Comparison versus (a) Edge GPU (b) Server GPU

Energy Efficiency. To demonstrate that EXION is an energy-efficient solution, we compared its energy efficiency with GPUs. Specifically, the experiments were conducted using batch sizes of one and eight, and ablation studies were performed to analyze the effect of each optimization method that EXION introduces. Figure 18 (a) shows the comparison results of EXION⁴ versus the edge GPU, where large models are not considered since executing them on an edge GPU is infeasible due to insufficient memory size. The experimental results with batch size one reveal that EXION⁴_Base achieves 0.5-31.4 TOPS/W energy efficiency within benchmarks, which is 91.1-2232.6 $\times$ more energy efficient than the edge GPU. Utilizing the resulting intra-iteration output sparsity by the EP method enables the efficiency gain to increase to 114.7-2401.7 $\times$. Notably, leveraging the inter-iteration output sparsity achieved by FFN-Reuse excels this gain, increasing it by 145.0-4347.2 $\times$. This reveals that optimizing the FFN layers in diffusion models is crucial. Finally, by utilizing both output sparsity types present in diffusion models, EXION⁴_All achieves 1.0-67.8 TOPS/W energy efficiency, which is 196.9-4668.2 $\times$ more energy efficient than the edge GPU.

Figure 18 (b) shows the comparison results of EXION²⁴ versus the server GPU. While most of the results show that the

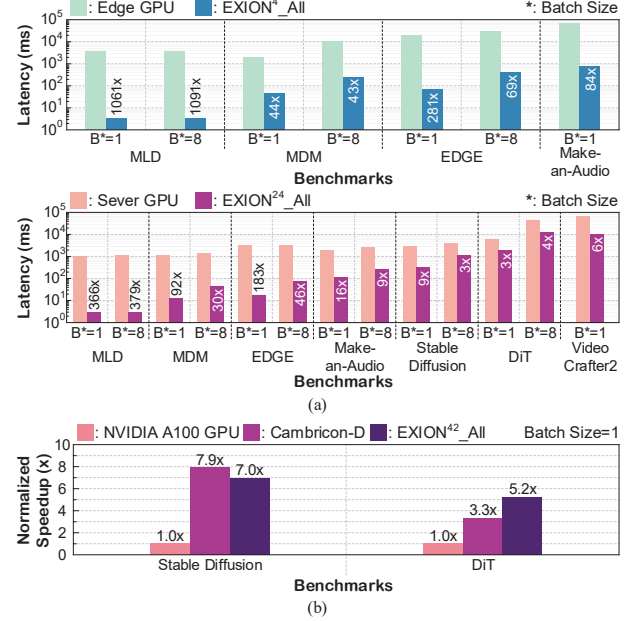


Fig. 19. (a) Latency Comparison versus Edge GPU and Server GPU and (b) Comparison of Speedup Over Different Accelerators

energy efficiency of EXION is significantly higher than that of the server GPU, its efficiency gain tends to drop in models such as Make-an-Audio and Stable Diffusion. This reduction is due to the presence of ResBlocks in these models, where we have not utilized any sparsity optimizations. However, considering the recent trend in ML research towards developing transformer-only architectures, the efficiency of EXION is still assured. Finally, EXION²⁴_All achieves 0.2-38.1 TOPS/W energy efficiency with batch size one, which is 45.1-3067.6 $\times$ more energy efficient than the server GPU, proving that the scale-out version of the EXION architecture is also energy-efficient and can be a promising solution for large models typically executed in servers. Additionally, the experimental results in Figures 18 (a) and (b) with batch size eight show that EXION is more efficient than the edge and server GPUs regardless of batch size, verifying its effectiveness in various inference settings.

Latency. Figure 19 (a) shows that the proposed EXION successfully reduces the long latency of diffusion models by utilizing inter- and intra-iteration output sparsity. The experiment compares the latency of EXION⁴_All and EXION²⁴_All with the edge GPU and server GPU, respectively, using two different batch sizes: one and eight. With batch size one, the speedup of EXION⁴_All and EXION²⁴_All achieves 43.7-1060.6 $\times$ and 3.3-365.6 $\times$, respectively. Notably, even with batch size eight, they achieve 42.6-1090.9 $\times$ and 3.2-379.3 $\times$ speedup over GPUs, respectively. This demonstrates that utilizing output sparsity through the ConMerge mechanism can dramatically reduce execution time in various settings.

Comparison with SOTA Accelerator. Figure 19 (b) compares the speedup of EXION and Cambricon-D [16], a SOTA

TABLE III
BREAKDOWN OF POWER AND AREA USAGE

Hardware Components	Area [mm ²]	Power [mW] [†]
SDUE	1.35	957.97
CAU	0.04	16.03
EPRE	0.81	265.15
CFSE	0.32	160.61
On-Chip Memories	1.79	60.41
Top Controller, DMA, Etc.	0.06	51.27
Total	4.37	1511.43

[†]: Measured at 800MHz, 0.8V

diffusion accelerator, over the NVIDIA A100 GPU [25]. Cambricon-D, specialized for accelerating image generation tasks, uses differential acceleration [22] on convolutional layers with a systolic array-based design to improve latency and energy efficiency through optimized memory access. We compare EXION⁴²_All, with 42 DSCs and a memory bandwidth of 1935 GB/s, to Cambricon-D, as both have similar throughput and memory bandwidth. For Stable Diffusion, which includes convolutional layers, Cambricon-D achieves a speedup of 7.9 $\times$, slightly exceeding EXION⁴²_All's speedup of 7.0 $\times$. However, for the DiT model, which lacks convolutional layers in the denoising iterations, EXION⁴²_All achieves a speedup of 5.2 $\times$, outperforming Cambricon-D's 3.3 $\times$ speedup. This result highlights our solution's ability to exploit output sparsity within transformer blocks.

It's worth noting that most SOTA image generation models [3], [30], [52] are transitioning from networks including ResBlocks towards transformer-block architectures, which limits the applicability of Cambricon-D's acceleration. Additionally, the optimizations in EXION can be further applied to other tasks, such as text- or audio-to-motion generations, where transformer blocks dominate the operations, positioning EXION as a versatile diffusion accelerator.

D. Power and Area Breakdown

Table III breaks down the power consumption and area of the EXION equipped with a single DSC. Comparison with the server GPU verifies that the architecture of EXION is area-efficient. Specifically, the area of EXION²⁴, which contains 24 DSCs and 64MB GSC, occupies 152.28mm². This is relatively smaller compared to the server GPU with a die area of 609mm² [42]. In terms of power consumption, EXION with a single DSC consumes 1511.43mW in total. Considering the peak power consumption of the server GPU and edge GPU are 300W and 15W, respectively, the power consumption of EXION is significantly lower even when scaled out. Furthermore, given the dramatic reduction in energy consumption thanks to utilizing output sparsity, it is acceptable that additional HW components such as EPRE and CAU for sparsity handling consume up to 18.6% of the total power.

VI. RELATED WORKS

Software-Based Approaches. A few SW-based approaches [6], [19], [36], [39] propose fast sampling methods that reduce the number of iterations. However, without retraining, the reduction is limited in achieving acceptable sampling quality [53]. Δ -DiT [4] suggests skipping computations of

certain DiT blocks by reusing cached output data across iterations. Despite these advancements, as these methods rely on GPU-based optimizations, they have not leveraged the opportunities presented by the significant yet unstructured inter- and intra-iteration sparsity. EXION successfully exploits them by proposing the ConMerge mechanism along with a dedicated HW architecture.

Transformer Accelerators. Recently, a large body of work [8], [9], [20], [33], [48] has been proposed to accelerate attention computation in transformer blocks, primarily targeting language models. However, FFN layers, which have the largest number of operations in recent diffusion models, have not yet been adequately addressed. Although methods handling FFN layers like FACT [32], which reduces energy consumption via mixed-precision, and SpAtten [47], which reduces computations via token pruning, can be applied to diffusion models, the high sparsity present in diffusion models is not yet employed. EXION achieves inter-iteration sparsity by reusing output data across iterations and directly translates this high sparsity into improvements in energy efficiency and performance.

SpGEMM Accelerators. Various generalized sparse matrix-matrix multiplication (SpGEMM) accelerators [10], [18], [29], [40], [50], [51] have been proposed to exploit sparsity present in MMUL operations. With the optimization of datapaths and hardware for sparsity handling, they can accelerate SpGEMM operations. SIGMA [31] and Trapezoid [49] propose methods to handle both sparse and dense operations by placing additional networks between memory and computational engines. However, the unique output sparsity present in diffusion models has yet to be addressed. EXION solves this problem by introducing the ConMerge mechanism and a dedicated hardware architecture that efficiently broadcasts both input and weight data.

VII. CONCLUSION

In this work, we present EXION, the first software-hardware co-designed diffusion accelerator that solves the computation challenges of excessive iterations by exploiting the unique inter- and intra-iteration output sparsity in diffusion models. To this end, we propose two software-level optimizations. First, we propose the FFN-Reuse algorithm that identifies and skips redundant computations in FFN layers across different iterations. Second, we use a modified eager prediction method that employs two-step leading-one detection to predict the attention score accurately. To handle the resulting unstructured sparsity, we introduce the data compaction mechanism, ConMerge, to condense and merge sparse matrices into compact forms. EXION's dedicated architecture supports this, translating high output sparsity into improved energy efficiency and performance. Our evaluation shows that EXION achieves significant improvements in performance and energy efficiency, with gains of up to 379.3 $\times$ and 3067.6 $\times$ over a server GPU and up to 1090.9 $\times$ and 4668.2 $\times$ over an edge GPU.

ACKNOWLEDGEMENTS

This work was supported in part by the Ministry of Science and ICT (MSIT), South Korea, under the Institute of Information & Communications Technology Planning & Evaluation (IITP) grants (No. 2022-0-01037, Development of High Performance Processing-In-Memory Technology based on DRAM) and (No. IITP-2025-RS-2023-00256472, the Graduate School of Artificial Intelligence Semiconductor), and by Samsung Electronics.

REFERENCES

- [1] T. Brooks, B. Peebles, C. Holmes, W. DePue, Y. Guo, L. Jing, D. Schnurr, J. Taylor, T. Luhman, E. Luhman, C. Ng, R. Wang, and A. Ramesh, *Video generation models as world simulators*, 2024. [Online]. Available: <https://openai.com/research/video-generation-models-as-world-simulators>
- [2] H. Chen, Y. Zhang, X. Cun, M. Xia, X. Wang, C. Weng, and Y. Shan, "Videocrafter2: Overcoming data limitations for high-quality video diffusion models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 7310–7320.
- [3] J. Chen, J. Yu, C. Ge, L. Yao, E. Xie, Y. Wu, Z. Wang, J. Kwok, P. Luo, H. Lu *et al.*, "Pixart- α : Fast training of diffusion transformer for photorealistic text-to-image synthesis," *arXiv preprint arXiv:2310.00426*, 2023.
- [4] P. Chen, M. Shen, P. Ye, J. Cao, C. Tu, C.-S. Bouganis, Y. Zhao, and T. Chen, " δ -dit: A training-free acceleration method tailored for diffusion transformers," *arXiv preprint arXiv:2406.01125*, 2024.
- [5] X. Chen, B. Jiang, W. Liu, Z. Huang, B. Fu, T. Chen, and G. Yu, "Executing your commands via motion diffusion in latent space," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 18 000–18 010.
- [6] H. Chung, B. Sim, and J. C. Ye, "Come-closer-diffuse-faster: Accelerating conditional diffusion models for inverse problems through stochastic contraction," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 413–12 422.
- [7] B. Elizalde, S. Deshmukh, M. Al Ismail, and H. Wang, "Clap learning audio concepts from natural language supervision," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
- [8] T. J. Ham, S. J. Jung, S. Kim, Y. H. Oh, Y. Park, Y. Song, J.-H. Park, S. Lee, K. Park, J. W. Lee *et al.*, "A³: Accelerating attention mechanisms in neural networks with approximation," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 328–341.
- [9] T. J. Ham, Y. Lee, S. H. Seo, S. Kim, H. Choi, S. J. Jung, and J. W. Lee, "Elsa: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 692–705.
- [10] K. Hegde, H. Asghari-Moghaddam, M. Pellauer, N. Crago, A. Jaleel, E. Solomonik, J. Emer, and C. W. Fletcher, "Extensor: An accelerator for sparse tensor algebra," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, 2019, pp. 319–333.
- [11] D. Hendrycks and K. Gimpel, "Gaussian error linear units (gelus)," *arXiv preprint arXiv:1606.08415*, 2016.
- [12] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.
- [13] R. Huang, J. Huang, D. Yang, Y. Ren, L. Liu, M. Li, Z. Ye, J. Liu, X. Yin, and Z. Zhao, "Make-an-audio: Text-to-audio generation with prompt-enhanced diffusion models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 13 916–13 932.
- [14] J. Kim, "The future of graphic and mobile memory for new applications," in *2016 IEEE Hot Chips 28 Symposium (HCS)*. IEEE Computer Society, 2016, pp. 1–25.
- [15] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer architecture letters*, vol. 15, no. 1, pp. 45–49, 2015.
- [16] W. Kong, Y. Hao, Q. Guo, Y. Zhao, X. Song, X. Li, M. Zou, Z. Du, R. Zhang, C. Liu *et al.*, "Cambricon-d: Full-network differential acceleration for diffusion models," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 903–914.
- [17] S. Lee, Y. Ro, Y. H. Son, H. Cho, N. S. Kim, and J. H. Ahn, "Understanding power-performance relationship of energy-efficient modern dram devices," in *2017 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 2017, pp. 110–111.
- [18] Z. Li, J. Li, T. Chen, D. Niu, H. Zheng, Y. Xie, and M. Gao, "Spada: Accelerating sparse matrix multiplication with adaptive dataflow," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2023, pp. 747–761.
- [19] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, "Dpm-solver++: Fast solver for guided sampling of diffusion probabilistic models," *arXiv preprint arXiv:2211.01095*, 2022.
- [20] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, "Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 977–991.
- [21] S. Luo, Y. Tan, S. Patil, D. Gu, P. von Platen, A. Passos, L. Huang, J. Li, and H. Zhao, "Lcm-lora: A universal stable-diffusion acceleration module," *arXiv preprint arXiv:2311.05556*, 2023.
- [22] M. Mahmoud, K. Siu, and A. Moshovos, "Diffy: A déjà vu-free differential deep neural network accelerator," in *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2018, pp. 134–147.
- [23] NVIDIA. Denoising diffusion-based generative modeling: Foundations and applications. [Online]. Available: <https://cvpr2022-tutorial-diffusion-models.github.io/>
- [24] NVIDIA. Jetson orin nano developer kit getting started guide. [Online]. Available: <https://developer.nvidia.com/embedded/learn/get-started-jetson-orin-nano-devkit>
- [25] NVIDIA. Nvidia a100 tensor core gpu. [Online]. Available: <https://www.nvidia.com/en-us/data-center/a100/>
- [26] NVIDIA. Nvidia rtx 6000 ada generation graphics card. [Online]. Available: <https://www.nvidia.com/en-us/design-visualization/rtx-6000/>
- [27] NVIDIA. System management interface smi. [Online]. Available: <https://developer.nvidia.com/system-management-interface>
- [28] NVIDIA. tegrastats utility. [Online]. Available: https://docs.nvidia.com/drive/drive_os_5.1.6.1L/nvlib_docs/index.html#page/DRIVE_OS_Linux_SDK_Development_Guide/Utilities/util_tegrastats.html
- [29] S. Pal, J. Beaumont, D.-H. Park, A. Amarnath, S. Feng, C. Chakrabarti, H.-S. Kim, D. Blaauw, T. Mudge, and R. Dreslinski, "Outerspace: An outer product based sparse matrix multiplication accelerator," in *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2018, pp. 724–736.
- [30] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 4195–4205.
- [31] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, "Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 58–70.
- [32] Y. Qin, Y. Wang, D. Deng, Z. Zhao, X. Yang, L. Liu, S. Wei, Y. Hu, and S. Yin, "Fact: Ffn-attention co-optimized transformer architecture with eager correlation prediction," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–14.
- [33] Z. Qu, L. Liu, F. Tu, Z. Chen, Y. Ding, and Y. Xie, "Dot2: detect and omit weak attentions for scalable transformer acceleration," in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 14–26.
- [34] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.*, "Learning transferable visual models from natural language supervision," in *International conference on machine learning*. PMLR, 2021, pp. 8748–8763.
- [35] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 10 684–10 695.
- [36] T. Salimans and J. Ho, "Progressive distillation for fast sampling of diffusion models," *arXiv preprint arXiv:2202.00512*, 2022.

- [37] A. Sauer, K. Schwarz, and A. Geiger, "Stylegan-xl: Scaling stylegan to large diverse datasets," in *ACM SIGGRAPH 2022 conference proceedings*, 2022, pp. 1–10.
- [38] N. Shazeer, "Glu variants improve transformer," 2020. [Online]. Available: <https://arxiv.org/abs/2002.05202>
- [39] Y. Song, P. Dhariwal, M. Chen, and I. Sutskever, "Consistency models," *arXiv preprint arXiv:2303.01469*, 2023.
- [40] N. Srivastava, H. Jin, J. Liu, D. Albonesi, and Z. Zhang, "Matraptor: A sparse-sparse matrix multiplication accelerator based on row-wise product," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 766–780.
- [41] Synopsys. synopsys design complier. [Online]. Available: <https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html>
- [42] Techpowerup. Rtx 6000 ada generation specs. [Online]. Available: <https://www.techpowerup.com/gpu-specs/rtx-6000-ada-generation.c3933>
- [43] G. Tevet, S. Raab, B. Gordon, Y. Shafir, D. Cohen-or, and A. H. Bermano, "Human motion diffusion model," in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=SJ1kSyO2jwu>
- [44] J. Tseng, R. Castellon, and K. Liu, "Edge: Editable dance generation from music," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 448–458.
- [45] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [46] C. S. Wallace, "A suggestion for a fast multiplier," *IEEE Transactions on electronic Computers*, no. 1, pp. 14–17, 1964.
- [47] H. Wang, Z. Zhang, and S. Han, "Spatten: Efficient sparse attention architecture with cascade token and head pruning," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 97–110.
- [48] Y. Wang, Y. Qin, D. Deng, J. Wei, Y. Zhou, Y. Fan, T. Chen, H. Sun, L. Liu, S. Wei *et al.*, "An energy-efficient transformer processor exploiting dynamic weak relevances in global attention," *IEEE Journal of Solid-State Circuits*, vol. 58, no. 1, pp. 227–242, 2022.
- [49] Y. Yang, J. S. Emer, and D. Sanchez, "Trapezoid: A versatile accelerator for dense and sparse matrix multiplications," in *Proceedings of the 51th Annual International Symposium on Computer Architecture*. IEEE, 2024, pp. 931–945.
- [50] G. Zhang, N. Attaluri, J. S. Emer, and D. Sanchez, "Gamma: Leveraging gustavson's algorithm to accelerate sparse matrix multiplication," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2021, pp. 687–701.
- [51] Z. Zhang, H. Wang, S. Han, and W. J. Dally, "Sparch: Efficient architecture for sparse matrix multiplication," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2020, pp. 261–274.
- [52] H. Zheng, W. Nie, A. Vahdat, and A. Anandkumar, "Fast training of diffusion models with masked transformers," *arXiv preprint arXiv:2306.09305*, 2023.
- [53] H. Zheng, W. Nie, A. Vahdat, K. Aizzadenesheli, and A. Anandkumar, "Fast sampling of diffusion models via operator learning," in *International conference on machine learning*. PMLR, 2023, pp. 42 390–42 402.